

Research Notes on Flash attention

Flash attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

>> Section 1

Alternative attention graphs The standard attention graph is either all-to-all or causal, both of which scales as $O(N^2)$ where N is the number of tokens in a sequence. Reformer (2020) reduces the computational load from $O(N^2)$ to $O(N \ln N)$ by using locality-sensitive hashing and reversible layers. Sparse attention uses attention graphs that grows slower than $O(N^2)$. For example, BigBird (2020) uses random small-world networks which grows as $O(N)$. Ordinary transformers require a memory size that is quadratic in the size of the context window. Attention-free transformers reduce this to a linear dependence while still retaining the advantages of a transformer by linking the key to the value.

>> Section 2

One set of (W^Q, W^K, W^V) matrices is called an attention head, and each layer in a transformer model has multiple attention heads. While each attention head attends to the tokens that are relevant to each token, multiple attention heads allow the model to do this for different definitions of "relevance". Specifically, the query and key projection matrices, W^Q and W^K , which are involved in the attention score computation, defines the "relevance". Meanwhile, the value projection matrix W^V , in combination with the part of the output projection matrix W^O , determines how the attended tokens influence what information is passed to subsequent layers and ultimately the output logits. In addition, the scope of attention, or the range of token relationships captured by each attention head, can expand as tokens pass through successive layers. This allows the model to capture more complex and long-range dependencies in deeper layers. Many transformer attention heads encode relevance relations that are meaningful to humans. For example, some attention heads can attend mostly to the next word, while others mainly attend from verbs to their direct objects. The computations for each attention head can be performed in parallel, which allows for fast processing. The outputs for the attention layer are concatenated to pass into the feedforward neural network layers. Concretely, let the multiple attention heads be indexed by i , then we have

$\text{MultiheadAttention}(Q, K, V) = \text{Concat}_{i \in [n \text{ heads}]} (\text{Attention}(XW_i^Q, XW_i^K, XW_i^V))W^O$ where the matrix X is the concatenation of word embeddings, and the matrices W_i^Q, W_i^K, W_i^V are "projection matrices" owned by individual attention head i , and W^O is a final projection matrix owned by the whole multihead attention head. It is theoretically possible for each attention head to have a different head dimension d_{head} , but that is rarely the case in practice. As an example, in the smallest GPT-2 model, there are only self-attention mechanisms. It has the following dimensions: $d_{\text{emb}} = 768, n_{\text{head}} = 12, d_{\text{head}} = 64$. Since $12 \times 64 = 768$, its output projection matrix $W^O \in \mathbb{R}^{(12 \times 64) \times 768}$ is a square matrix.

>> Section 3

RoPE (rotary positional embedding), is best explained by considering a list of 2-dimensional vectors $((x_1^{(1)}, x_1^{(2)}), (x_2^{(1)}, x_2^{(2)}), (x_3^{(1)}, x_3^{(2)}), \dots)$. Now pick some angle θ . Then RoPE encoding is $\text{RoPE}(x_m^{(1)}, x_m^{(2)}, m) = (\cos m\theta - \sin m\theta \sin m\theta, \sin m\theta \cos m\theta, x_m^{(1)} \cos m\theta - x_m^{(2)} \sin m\theta, x_m^{(2)} \cos m\theta + x_m^{(1)} \sin m\theta)$. Equivalently, if we write the 2-dimensional vectors as complex numbers $z_m := x_m^{(1)} + ix_m^{(2)}$, then RoPE encoding is just multiplication by an angle: $\text{RoPE}(z_m, m) = e^{im\theta} z_m$. For a list of $2n$ -dimensional vectors, a RoPE encoder is defined by a sequence of angles $\theta^{(1)}, \dots, \theta^{(n)}$. Then the RoPE encoding is applied to each pair of coordinates. The benefit of RoPE is that the dot-product between two vectors depends on their relative location only: $\text{RoPE}(x, m)^T \text{RoPE}(y, n) = \text{RoPE}(x, m+k)^T \text{RoPE}(y, n+k)$

Research Notes on Flash attention

Flash attention

$$\begin{aligned} & \left(\left(x, m \right) \right)^{\text{RoPE}} \left(y, n \right) \\ &= \left(\left(x, m+k \right) \right)^{\text{RoPE}} \left(y, n+k \right) \end{aligned}$$

>> Section 4

Applications The transformer has had great success in natural language processing (NLP). Many large language models such as GPT-2, GPT-3, GPT-4, Gemini, AlbertAGPT, Claude, BERT, Grok, XLNet, RoBERTa and ChatGPT demonstrate the ability of transformers to perform a wide variety of NLP-related subtasks and their related real-world applications, including:

>> Section 5

ALiBi ALiBi (Attention with Linear Biases) is not a replacement for the positional encoder on the original transformer. Instead, it is an additional positional encoder that is directly plugged into the attention mechanism. Specifically, the ALiBi attention mechanism is Attention (Q, K, V) = softmax($\frac{QK^T}{\sqrt{d_k}} + B$) V . Here, s is a real number ("scalar"), and B is the linear bias matrix defined by $B = \begin{pmatrix} 0 & 1 & 2 & 3 & \dots & -1 & 0 & 1 & 2 & \dots & -2 \\ -1 & 0 & 1 & 2 & \dots & -3 & -2 & -1 & 0 & \dots & -\infty \end{pmatrix}$ in other words, $B_{i,j} = j - i$. The idea being that the linear bias matrix is a softened mask. Just as 0 represent full attention paid, and $-\infty$ represents no attention paid, the linear bias matrix increases attention paid in one direction and decreases attention paid in the other direction. ALiBi allows pretraining on short context windows, then fine-tuning on longer context windows. Since it is directly plugged into the attention mechanism, it can be combined with any positional encoder that is plugged into the "bottom" of the entire network (which is where the sinusoidal encoder on the original transformer, as well as RoPE and many others, are located).